

Lecture 10

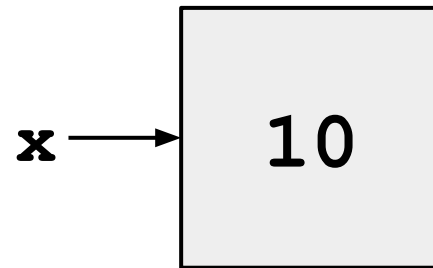
Mutation and Aliasing

References

- New types: **t ref** where **t** is a type
 - References are first-class
- New expressions:
 - **ref e** to create a reference with initial contents **e**
 - Just a record with one *mutable* field **contents**
 - **e1 := e2** to update contents
 - **!e** to retrieve contents (not negation)
- Aliasing matters for references because of mutability

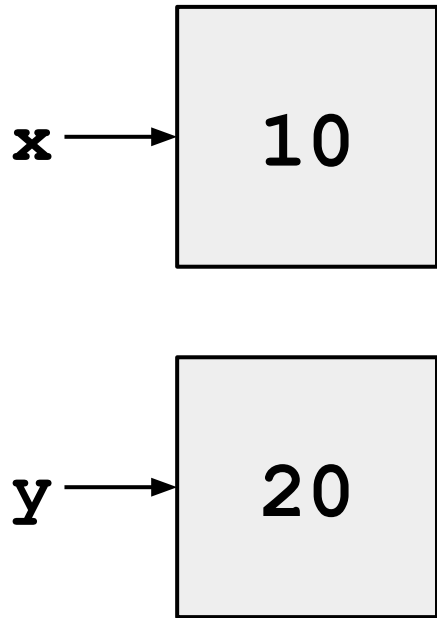
References example

```
let x = ref 10 (* : int ref *)
```



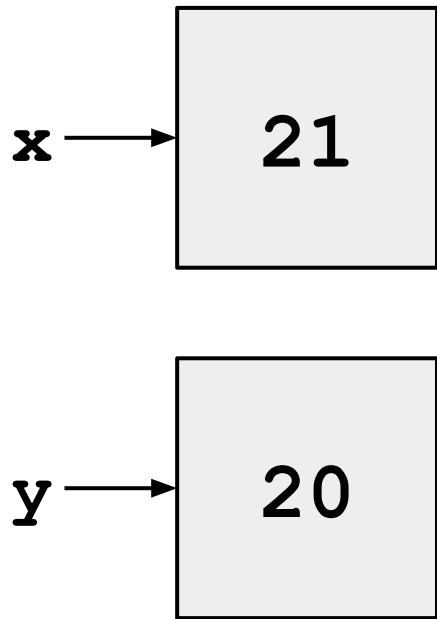
References example

```
let x = ref 10 (* : int ref *)  
let y = ref 20 (* : int ref *)
```



References example

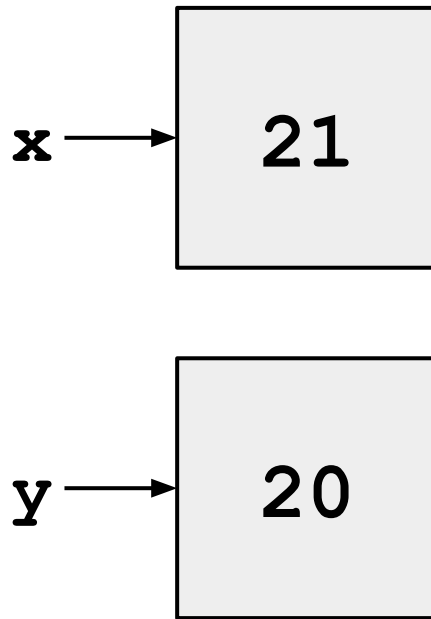
```
let x = ref 10 (* : int ref *)  
let y = ref 20 (* : int ref *)  
let _ = x := !y + 1
```



References example

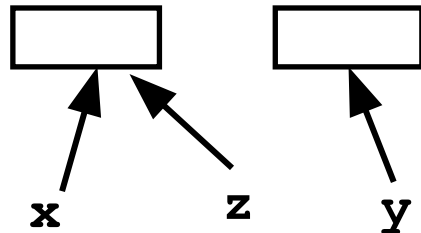
```
let x = ref 10 (* : int ref *)  
let y = ref 20 (* : int ref *)  
let _ = x := !y + 1
```

`:=` is an operator with type
`'a ref -> 'a -> unit`



References example

```
let x = ref 42 (* : int ref *)
let y = ref 42
let z = x
let _ = x := 43
let w = (!y) + (!z) (* 85 *)
(* x + 1 does not type-check *)
```



- A variable bound to a reference (e.g., `x`) is still immutable: it will always refer to the *same reference*
- But the *contents* of the reference may change via `:=`
- And there may be aliases to the reference, which matter a lot
- References are first-class values

Group Work

For each new expression type:

- `ref e`
- `! e`
- `e1 := e2`

Write out the 341 recipe:

- Type checking rules
- Evaluation rules

```
let x = ref 42 (* : int ref *)
let y = ref 42
let z = x
let _ = x := 43
let w = (!y) + (!z) (* 85 *)
(* x + 1 does not type-check *)
```


The Recipe

Syntax	ref <i>e</i>	! <i>e</i>	<i>e</i> 1 := <i>e</i> 2
Type Checking	if <i>e</i> has type <i>t</i> then ref <i>e</i> has type <i>t ref</i>	if <i>e</i> has type <i>t ref</i> then ! <i>e</i> has type <i>t</i>	if <i>e</i> 1 has type <i>t ref</i> and <i>e</i> 2 has type <i>t</i> then <i>e</i> 1 := <i>e</i> 2 has type <i>unit</i>
Evaluation	Evaluate <i>e</i> to value <i>v</i> Create a fresh ref cell on the heap containing <i>v</i>	Evaluate <i>e</i> to a ref cell containing <i>v</i> ! <i>e</i> evaluates to <i>v</i>	Evaluate <i>e</i> 1 to a ref cell. Evaluate <i>e</i> 2 to a value <i>v</i> . Update the value in the ref cell to be <i>v</i> and return ()

Group Work

At each labeled program point, draw the references and their current values as boxes with arrows showing which variable points to which box

```
let incr r x = r := !r + x
let a = ref 0
let b = a
let c = ref 0
(* POINT 1 *)
let _ = incr c 10
let d = ref !c
(* POINT 2 *)
let _ = incr b 5
let _ = incr d 1
let _ = incr c !d
(* POINT 3 *)
```

Callbacks

A common idiom: Library takes functions to apply later, when an *event* occurs – examples:

- When a key is pressed, mouse moves, data arrives
- When the program enters some state (e.g., turns in a game)

A library may accept multiple callbacks

- Different callbacks may need different private data with different types
- Fortunately, a function's type does not include the types of bindings in its environment
- (In OOP, objects and private fields are used similarly, e.g., Java “listeners”)

Mutable state

While it's not absolutely necessary, mutable state is reasonably appropriate here...

... We really do want the “collection of callbacks registered” to *change* when a function to register a callback is called

Example call-back library

Library maintains mutable state for “what callbacks are there” and provides a function for accepting new ones

- A real library would also support removing them, etc.
- In example, callbacks have type `int->unit`

So the entire public library interface would be the function for registering new callbacks:

```
val onKeyEvent : (int -> unit) -> unit
```

(Because callbacks are executed for side-effect, they may also need mutable state)

Library implementation

```
let callbacks : (int -> unit) list ref = ref []
```

```
let on_key_event f =  
  callbacks := f :: !callbacks
```

```
let do_key_event i =  
  List.iter (fun f -> f i) !callbacks
```

Clients

Can only register an `int -> unit`, so if any other data is needed, must be in closure's environment

- And if need to “remember” something, can use mutable state

```
let times_pressed = ref 0
let _ = on_key_event (fun _ ->
    times_pressed := !times_pressed + 1
let print_if_pressed i =
    on_key_event (fun j ->
        if i = j
        then print_endline ("pressed " ^ string_of_int i)
        else ()))
```

Aliasing

Can you spot the difference? Can a client?

```
let sort_pair (pr : int*int)=  
  if fst pr < snd pr then  
    pr  
  else  
    (snd pr, fst pr)
```

```
let sort_pair (pr : int*int)=  
  if fst pr < snd pr then  
    (fst pr, snd pr)  
  else  
    (snd pr, fst pr)
```

Can you spot the difference? Can a client?

```
let sort_pair (pr : int*int)=  
  if fst pr < snd pr then  
    pr  
  else  
    (snd pr, fst pr)
```

```
let sort_pair (pr : int*int)=  
  if fst pr < snd pr then  
    (fst pr, snd pr)  
  else  
    (snd pr, fst pr)
```

- If pair already sorted, does this return:
 - (an alias to) the same pair, or
 - a new pair with the same contents?
- In OCaml, *we cannot tell* which way `sort_pair` is implemented (!!)
 - This “*negative expressiveness*” is a big deal!
 - When data is immutable, aliasing does not matter

Can you spot the difference? Can a client?

```
let sort_pair (pr : int*int)=  
  if fst pr < snd pr then  
    pr  
  else  
    (snd pr, fst pr)
```

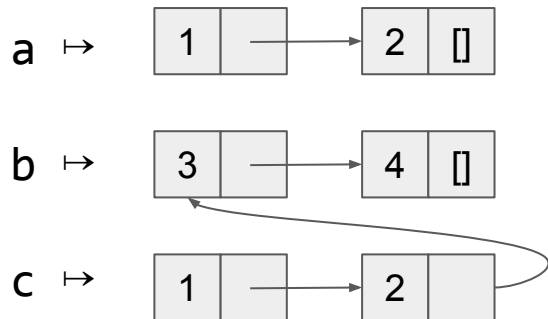
```
let sort_pair (pr : int*int)=  
  if fst pr < snd pr then  
    (fst pr, snd pr)  
  else  
    (snd pr, fst pr)
```

- If pair already sorted, does this return:
 - (an alias to) the same pair, or
 - a new pair with the same contents?
- In OCaml, *we cannot tell* which way `sort_pair` is implemented (!!)
 - This “*negative expressiveness*” is a big deal!
 - When data is immutable, aliasing does not matter
- What if we had `(int * int) ref` instead?

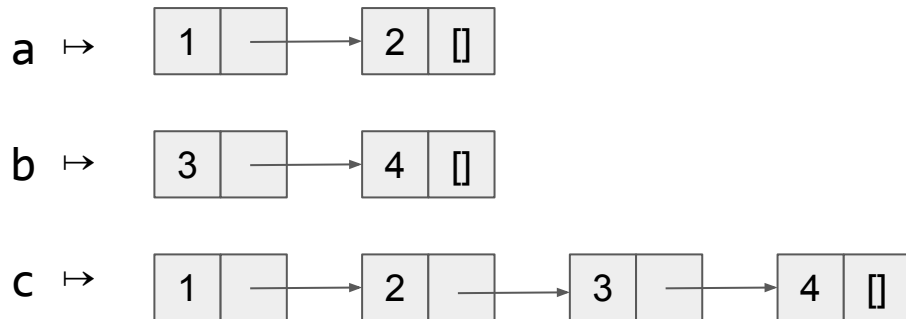
Aliasing with append

```
let rec append ((xs:'a list), (ys:'a list)) =  
  if xs = []  
  then ys  
  else List.hd xs :: append (List.tl xs, ys)  
let a = [1; 2]  
let b = [3; 4]  
let c = append (a,b)
```

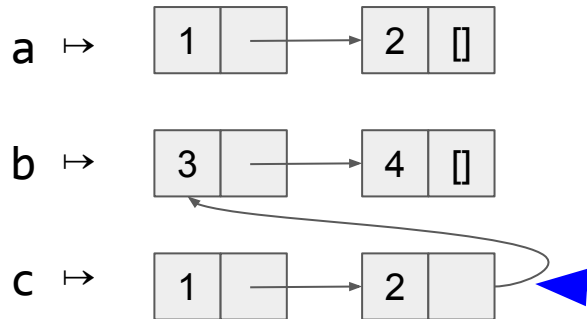
Aliasing with append



```
let rec append ((xs:'a list), (ys:'a list)) =  
  if xs = []  
  then ys  
  else List.hd xs :: append (List.tl xs, ys)  
let a = [1; 2]  
let b = [3; 4]  
let c = append (a,b)
```

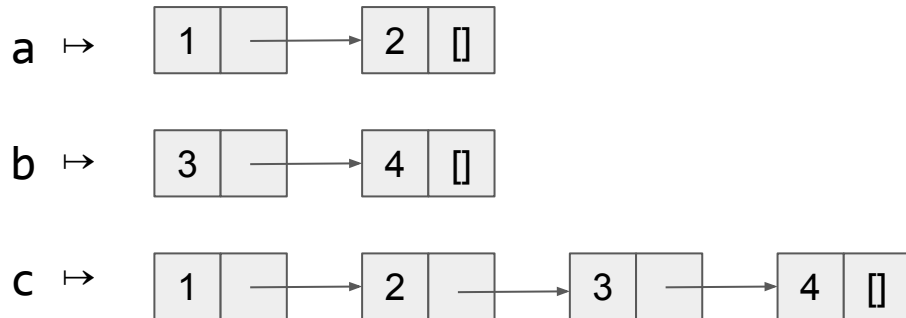


Aliasing with append



```
let rec append ((xs:'a list), (ys:'a list)) =  
  if xs = []  
  then ys  
  else List.hd xs :: append (List.tl xs, ys)  
let a = [1; 2]  
let b = [3; 4]  
let c = append (a,b)
```

Clients can never tell!
(but it's actually the upper one)



Can *safely* reuse and share data.
Immutability can be *more efficient*!

Immutability is great!

- In OCaml, we create aliases all the time without thinking about it
 - It is *impossible* to tell where there is aliasing
 - Example: `List.tl` is constant time; doesn't copy anything
 - No need to worry!
- In languages with mostly mutable data (e.g., Java), programmers are *obsessed* with aliasing and object identity
 - They have to be (!)
 - So that later assignments affect the right parts of the program
 - Often crucial to make copies in just the right places
 - Consider a Java example

Java security nightmare (bad code)

```
class ProtectedResource {
    private Resource theResource = ...;
    private String[] allowedUsers = ...;
    public String[] getAllowedUsers() {
        return allowedUsers;
    }
    public String currentUser() { ... }
    public void useTheResource() {
        for(int i=0; i < allowedUsers.length; i++) {
            if(currentUser().equals(allowedUsers[i])) {
                ... // access allowed: use it
                return;
            }
        }
        throw new IllegalAccessException();
    }
}
```


Have to make copies

The problem:*

```
p.getAllowedUsers()[0] = p.currentUser();  
p.useTheResource();
```

The fix:

```
public String[] getAllowedUsers() {  
    ... return a copy of allowedUsers ...  
}
```

Copying immutable data is unnecessary!

*CSE 331 calls this “representation exposure”

= versus ==

Equality in OCaml

- OCaml has two different equality operators: `=` and `==`
 - `e1 <> e2` is the same as `not (e1 = e2)`
 - `e1 != e2` is the same as `not (e1 == e2)`
- For CSE 341 always use `=` (or `<>`)
 - *Not* the character sequences you are used to ⚠
 - It turns out not to matter for ints and bools though
- The difference between them is interesting and relevant to aliasing...

Structural equality

- `=` is defined recursively: same data in same shape
- `3=3`, `(1,2)=(1,2)`, `[3;4;5]=[3;4;5]`, `None=None`, `Some 7 = Some 7`, etc.
- Two subexpressions must have same type
 - Hard to be equal otherwise anyway
- Notice:
 - If `x` and `y` are aliases, then `x=y` must evaluate to true
 - If `x` and `y` are not aliases, then `x=y` might evaluate to true or false
 - Aliasing doesn't matter
- More like Java's `equals`, but predefined by the language

Reference equality

- `e1 == e2` is true if `e1` and `e2` evaluate to aliases (or same `int`, `bool`, etc.)
 - For data structures, this is faster to check than structural equality
- But wait: then clients *can* tell if two things are aliases
 - Could complicate writing code and implementing the language
 - And you really shouldn't care about aliases for immutable data
- So OCaml reached a pragmatic compromise (hack?)
 - The “official definition” of `==` promises almost nothing for immutable data:
 - *all* it guarantees is `==` implies `=`
 - If you assume it means more, you assume the risk of surprise
 - Keeps “aliases” out of the language definition
 - a key advantage of immutable data!
- Again: do not use `==` (nor `!=`) in CSE 341